

V. ADDENDUM: GPU-ACCELERATED PPO WITH GRAPHGPS

This addendum details the engineering updates that complement the joint wireless and plug-in charger placement framework. The new workflow preserves the scenario realism of MATSim [5] while reducing turnaround time and improving reproducibility through a GPU-first reinforcement learning stack built on PPO [20], PyTorch [26], PyTorch Geometric [23], and Gymnasium [22].

A. System Enhancements

We restructured *contribs/rlev* into a standalone Maven module that emits a shaded *rlev-uber.jar*. The shaded jar pins MATSim 15.0, GeoTools 29.0, and other transitive dependencies, so MATSim can be launched without a parent build. A lightweight launcher rewrites scenario configs, removes obsolete counts modules, enables EV telemetry, and enforces single-iteration runs for PPO rollouts.

MATSim now executes inside the PPO loop through a JPyPy bridge [27]. The bridge initializes MATSim once per experiment, injects chargers per rollout, and collects rewards through an in-memory *RewardProbe*, avoiding the socket latency and serialization overhead of the legacy reward server.

Scenario assets are sandboxed for every rollout. *MatsimXMLDataset* copies the configuration into a temporary workspace, normalizes absolute paths, tolerates *.xml.gz* inputs, and regenerates plans and vehicles when synthetic demand is requested. Outputs are written to run-scoped folders (*output_rl/*), keeping the canonical network untouched while archiving TensorBoard logs, *best_chargers.csv* files, and zipped MATSim outputs.

B. Reward Extraction and Analytics

The local controller computes rewards directly from MATSim diagnostics. Charger rewards integrate energy delivered in *0.average_charge_time_profiles.txt* and normalize by fleet battery capacity, while time penalties derive from leg-duration summaries. When telemetry is missing, the controller log provides a fallback signal. The legacy HTTP server reuses the same probe and shaded jar, ensuring parity between distributed and local experiments.

Telemetry is now exportable: TensorBoard tracks charger efficiency, cost, reward, and temporal stability, and the best-scoring configurations are persisted for downstream visualization or validation. The callback hooks into Stable-Baselines3's logging interface after each rollout, reads immutable copies of statistics from the environment *info*, and emits summaries without mutating replay buffers or optimizer state, so the PPO optimisation path remains identical to the baseline. These diagnostics eliminate the manual bookkeeping that previously accompanied PPO training.

TensorBoard-Enabled PPO/OCP Architecture

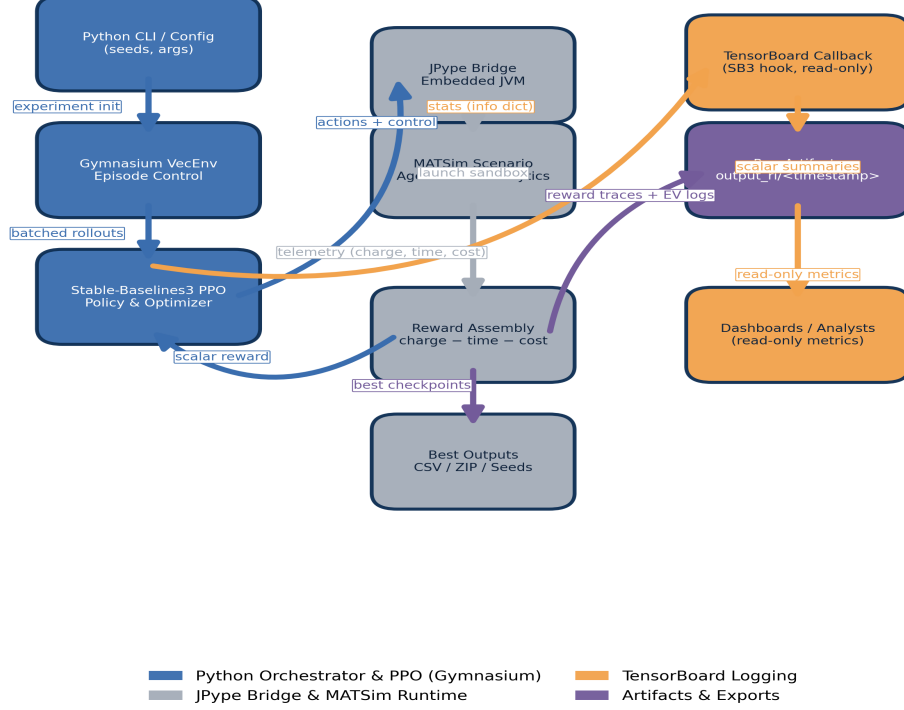


Fig. 5. Updated TensorBoard architecture. Left column (blue) shows Python + Gymnasium orchestrating PPO with MATSim. Centre (gray) captures embedded JVM execution and reward assembly. Right column (orange/violet) illustrates telemetry to TensorBoard and artifact export.

C. GraphGPS Observation Encoder

A new GraphGPS extractor [25] encodes MATSim networks into fixed-size graph representations. Each rollout selects the top-K line-graph nodes by degree, applies Graph Isomorphism Network layers with fp32-normalized residuals, and performs multi-head attention before global pooling. The encoder enables automatic mixed precision and optional CUDA graph capture, providing deterministic execution on NVIDIA hardware while clamping floating-point outliers to maintain numerical stability.

D. PPO Training Stack

The PPO runner relies on Stable-Baselines3 [20] but patches internal buffers so multi-discrete actions remain on GPU. Using *DummyVecEnv* sidesteps Windows process forking, and a comprehensive CLI exposes backend selection (*python*, *jvm*, or *server*), CUDA graph toggles, and GraphGPS hyperparameters. Each run records its arguments and aggregates per-step statistics, simplifying replication and ablation.

E. Operational Impact

The GPU-accelerated stack reduces per-iteration wall time to the MATSim runtime plus a modest constant, because graph extraction and reward computation now operate entirely in-process. Training no longer depends on 100 GB of RAM or multi-day execution windows; experiments complete in hours, and concurrent runs avoid file conflicts through sandboxing. These improvements maintain the fidelity of the original study while broadening its practicality.

REFERENCES

- [25] O. Rampášek, B. G. T. Touri, B. Barekatin, et al., "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, 2022.
- [26] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019.
- [27] JPytype Contributors, "JPytype User Guide," <https://jpytype.readthedocs.io/>, accessed Sep. 2025.